In common experience, binary heaps are never the fastest, 5-way heaps are seldom faster (for narrow domains), and the best choice usually falls between 3 and 4, depending on nuances.

So, while I feel safe to suggest starting with a branching factor 4, if this data structure is used in a key section of your application and small performance improvement can make a relevant difference, then you need to tune the branching factor as a parameter.

2.9.4 Branching factor vs memory

Notice that I suggested the larger branching factor among the two best performing ones for a reason; turns out there is, in fact, another consideration to be made when looking for the optimal branching factor for heaps: locality of reference.

When the size of the heap is larger than available cache, or than the available memory, or in any case where caching and multiple levels of storage are involved, then on average a binary heap requires more cache misses or page faults than a d-ary heap. Intuitively, this is due to the fact that children are stored in clusters, and that on insertion or delete, for every node reached, all its children will have to be examined. The largest the branching factor becomes, the more the heap becomes short and wide, the more the principle of locality applies.

D-way heap appears to be the best traditional data structure for reducing page faults[22]. New alternatives focused on